

TECHCATALYST DE 2026

Developer foundations

VS Code, GitHub Codespaces, Python environments, and a hands-on Git and GitHub deep dive, all set up and understood by hand.

The Hartford · TechCatalyst Data Engineering 2026

Week 1 Day 3

VS Code essentials

GitHub Codespaces

Python environments

venv · pip · conda · uv

Git & GitHub deep dive

Two hands-on labs

Markdown Mash Time!

Pop Quiz



DAY 3

Today's agenda

Time	Block
8:00 to 8:30	Recap and stand-up
8:30 to 9:15	Toolkit, AI-free zone, VS Code and Codespaces (demo)
9:15 to 10:00	Codespaces budget; Python envs: venv, pip, conda, uv (demo)
10:15 to 12:00	Lab A: your repo and your environment
1:00 to 2:30	Git deep dive: mental model through history (demo)
2:30 to 3:00	Git and the remote: push, pull, clone, fetch (demo)
3:00 to 4:30	Lab B: local meets remote (plus optional stretch)
4:30 to 5:00	Troubleshooting clinic, recap, exit ticket

GROUND RULES

Weeks 1 to 4 are an AI-free zone

“You cannot review code you could not have written.”

Until Week 6

- No Copilot, no LLM-generated code
- Write Python, SQL, and Git by hand
- Read error messages; debug first
- Docs, manuals, and peers are fair game

Why we do this

- Syntax fluency comes from typing it
- Debugging resilience is the #1 junior gap
- In Week 6, Copilot is a reviewer, not a crutch
- Employers test you without AI too

TOOLKIT

The toolkit map

VS Code

Your editor, local or in the browser

Codespaces

A dev machine in the cloud, attached to your repo

Terminal

Where real work happens (Week 2 deep dive)

Python + an env tool

Isolated, reproducible project environments

Git + GitHub

Version control and collaboration

Everything is text, files, and commands. Master these and every cloud tool gets easier.

01

VS Code and Codespaces

One editor, anywhere.

VS Code essentials

Must-know	What it does
Command palette (Ctrl/Cmd+Shift+P)	Run any command without hunting menus
Integrated terminal (Ctrl+`)	A shell inside the editor
Explorer & quick open (Ctrl/Cmd+P)	Jump to any file by name
Extensions	Python, Jupyter, GitLens: install today
Source Control panel	Stage, commit, and diff with buttons (same Git)

Demo: we tour each live, then you replicate it in the lab.

CODESPACES

GitHub Codespaces: your cloud dev machine



- Identical environment for everyone; nothing blocked by corporate machines
- A real Ubuntu machine: full terminal, Python, and Git
- Free tier: 120 core-hours per month per account, so stop it when done
- Fallbacks if blocked: Google Colab (Python only) or a provisioned Ubuntu VM

BUDGET

Codespaces: do not burn your free hours

120 core-hours and **15 GB storage** per month (personal free tier).

A 2-core Codespace burns the 120 hours in about 60 hours of active time; a 4-core in about 30.

- Stop a Codespace when you step away (idle timeout defaults to 30 min; do not rely on it)
- Stop keeps your state and still uses storage; delete frees storage but loses uncommitted work, so push first
- Check usage on the GitHub billing and plans page (Codespaces section)
- Core-hours bill on machine size times active time; storage bills on gigabyte-months

COST DISCIPLINE

Free-tier discipline across the program

Resource	Free allowance	The habit
GitHub Codespaces	120 core-hours, 15 GB / month	Stop when idle; push before deleting
GCP free tier and credits	Always-free products plus trial credit	Set a budget and alerts (Day 2)
BigQuery Sandbox	10 GB storage, 1 TB queried / month	Watch bytes scanned (Day 2)

One mindset: know the meter, stop what you are not using.

CODESPACES WRAP-UP

GitHub Codespaces: your cloud dev environment



Free tier (personal GitHub account)

120 core-hours / month

15 GB-month storage

A 2-core codespace can use those 120 core-hours in about 60 active hours.



Preconfigured

Same ready-to-use environment for everyone on the repo



Runs in the cloud

Uses remote compute instead of your laptop



Choose your editor

Browser, VS Code desktop, or JupyterLab



Multiple workspaces

Separate codespaces for projects or branches



Share & preview

Forward ports, test apps, and collaborate

Big idea: your repo can define the dev environment, so setup becomes faster, repeatable, and easier to teach.

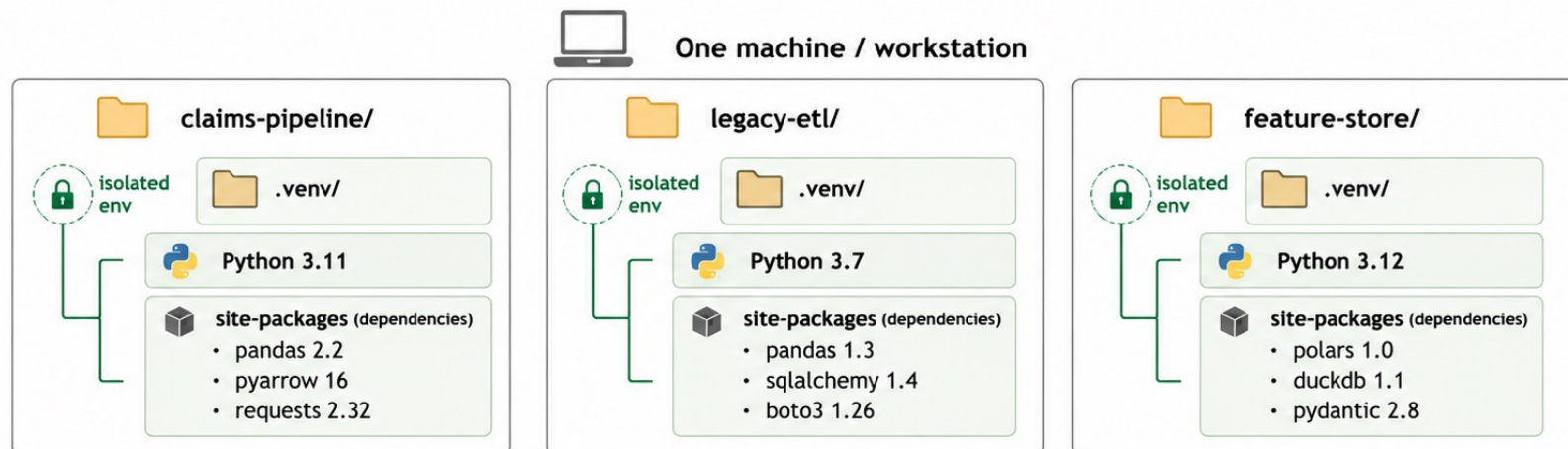
02

Python environments

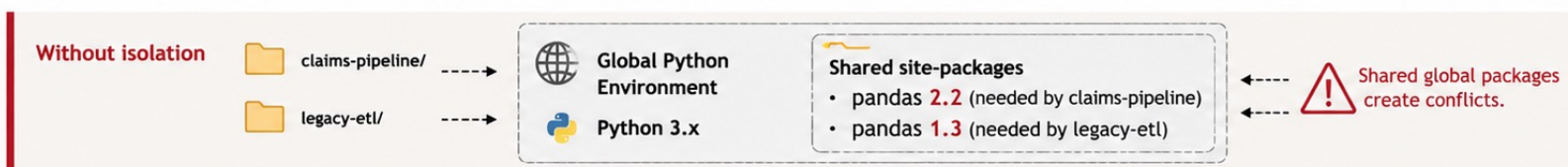
Reproducibility starts here.

PYTHON

Why isolate every project



Each project gets its own Python + dependency stack. No version conflicts.



Virtual environments make projects reproducible, safer to upgrade, and easier to share.

Why isolate every project

The problem: project A needs pandas 2.x; project B needs 1.5. Install both globally and they fight. The last install wins and something breaks.

The fix: one project, one isolated environment, one record of what is installed. No shared global packages.

The payoff: a teammate, a server, or future-you can recreate the exact environment from the record.

The Python tooling landscape

Tool	Env	Packages	When you reach for it
venv	Yes	No	Always available; the standard-library way to isolate
pip	No	Yes	Installs from PyPI into the active environment
conda	Yes	Yes	Non-Python / scientific binaries; data-science stacks
uv	Yes	Yes	One fast tool for env + packages + a lock file

Two jobs: isolate the environment, and manage the packages. Some tools do one, some do both. venv + pip together equal what conda or uv each do alone.

PYTHON

venv + pip + requirements.txt

```
# create an isolated environment
python -m venv .venv

# activate it (Linux/macOS)
source .venv/bin/activate

# install + record dependencies
pip install pandas requests
pip freeze > requirements.txt

# recreate it anywhere
pip install -r requirements.txt
```

The course baseline.

Three moving parts, all visible: venv isolates, pip installs, requirements.txt records.

The prompt shows (.venv) when active: make checking it a reflex.

Add .venv/ to .gitignore: environments are never committed.

conda: environments and packages

- Manages environments AND packages in one tool
- Installs non-Python binaries pip cannot (system libraries for geospatial, CUDA, scientific stacks)
- Packages come from channels; conda-forge is the big community channel
- Miniconda is the minimal installer; Anaconda is the large bundled distribution

Caution: mixing conda and pip works but needs care: install conda packages first, then pip, and do not double-install the same package.

```
conda create -n claims python=3.11 pandas # conda activate claims
conda install -c conda-forge pyarrow
```

uv: why it is gaining adoption

One fast tool (written in Rust) for environments and packages, with a lock file.

Why it is popular

- Speed: resolution and installs are far faster than pip (compiled and parallelized)
- One tool instead of several: replaces venv, pip, pip-tools, and parts of pipenv and poetry
- Reproducibility by default: a project gets a lock file pinning exact versions
- Drop-in friendly: uv pip install and uv venv mirror commands you already know

Key features

Project model with pyproject.toml · a lock file (uv.lock) · uv run to execute inside the env · manages Python versions too

PYTHON

uv workflow: add, sync, and the lock file

```
# start a project (pyproject.toml)
uv init claims-pipeline && cd ...

# add a dep: records + relocks
uv add pandas

# make env match the lock exactly
uv sync

# run inside the env, no activation
uv run python hello_data.py
```

add changes what the project depends on, and relocks: “I want this new thing.”

sync reconciles the installed env to the existing lock: “make this machine match.”

uv.lock pins exact versions of every direct and transitive dependency.

PYTHON

Conda vs uv vs venv + pip

 venv + pip The standard baseline	 conda Envs + packages, including non-Python binaries	 uv Fast modern workflow
 Environment management	 Environment management	 Environment management
Yes (venv)	Yes	Yes
 Package management	 Package management	 Package management
Yes (pip)	Yes	Yes
 Typical files	 Typical files	 Typical files
.venv requirements.txt	environment.yml	pyproject.toml uv.lock
 Best for	 Best for	 Best for
Most Python projects, teaching fundamentals, simple and widely available	Data science, geospatial, CUDA, scientific stacks, binary-heavy dependencies	Modern Python projects, fast installs, reproducible environments
 Strengths	 Strengths	 Strengths
Built in, easy to explain, works everywhere	Handles Python and non-Python packages; great for complex native dependencies	Very fast, one tool, lock file by default, convenient run/sync workflow
 Watch-outs	 Watch-outs	 Watch-outs
Two tools to manage; requirements.txt is less strict than a lock-first workflow	Uses channels; larger ecosystem footprint; mixing pip needs care	Newer tool; teams may still use older workflows

 Built-in?	Yes	No	No
 Non-Python packages?	No	Yes	Mostly Python-focused
 Lock file by default?	No	Not usually	Yes
 Best fit	Course baseline / general Python	Scientific / binary-heavy stacks	Modern fast workflow



Rule of thumb: venv + pip is the baseline, conda shines for binary-heavy stacks, and uv is the modern fast default for many new projects.

03

Git deep dive

Snapshots, not saves. Solo workflow today; teamwork in Week 2.

GIT

Git and GitHub are not the same

Tool	What it is	Where it runs
Git	A version control system	Your machine or Codespace
GitHub	Repository hosting and collaboration	The web

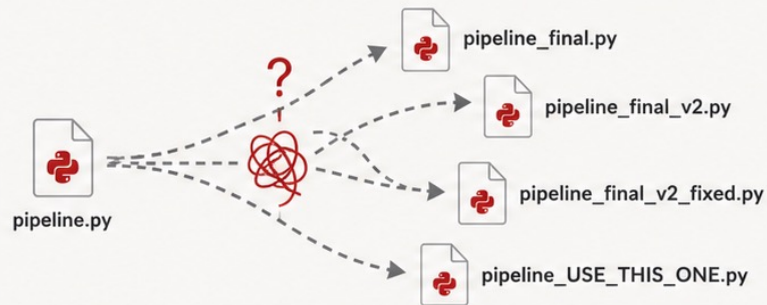
First habit: when unsure, run `git status` and read what Git tells you.

GIT

Why Git?

Without Git

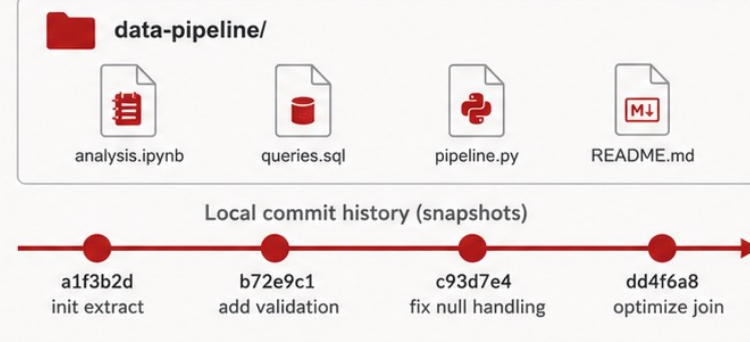
The “Save As” spiral — one project, many confusing copies



- ❌ Hard to know which file is latest
- ❌ No clean history of what changed
- ❌ Rollback means hunting through copies

With Git (local version control)

One project, clear history of snapshots



- ✅ One project folder
- ✅ Each commit is a named snapshot
- ✅ Compare or restore any earlier point
- ✅ Branch safely when experimenting



Git replaces messy duplicate filenames with a clear local history of snapshots.



Git works locally —
GitHub is optional.

GIT

The Git mental model: four areas

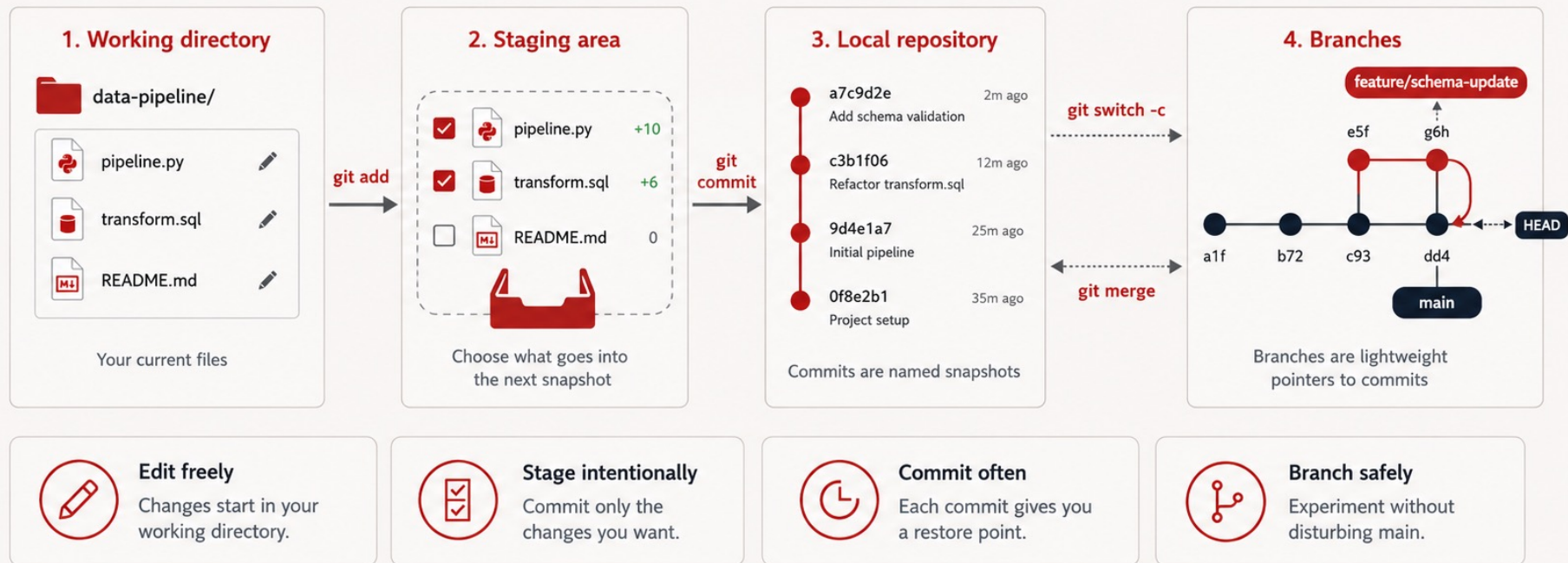


- A commit is a snapshot of your whole project with a message explaining why
- `git status` is your compass: run it constantly
- Today: init or clone, add, commit, push, pull. Week 2: branching and merges

GIT

Local Git lifecycle

Edit files, stage changes, create snapshots, and branch safely.



Think of Git locally as: *working files* → *staged changes* → *commit history* → *branches*.



Remote hosting (GitHub) is a separate step.

GIT

Start a repo; git status is your compass

```
mkdir claims-journal && cd claims-journal
git status      # error: not a git repository. Git opts IN per folder.
git init        # creates the hidden .git/ folder: the repository itself
ls -a           # see .git
git status      # On branch main, no commits yet
```

- Git never watches your whole computer; you tell it which folders to track
- The hidden .git/ folder IS the repository (its database of every version)
- Never edit or delete .git/ by hand: that is your history

GIT

The edit, stage, commit loop

```
echo "# Claims Journal" > README.md
git status                # README.md is untracked (red)
git add README.md
git status                # staged (green): same file, new area
git commit -m "Add claims journal README"
git status                # clean
```

- Red = working directory; green = staging area
- Nothing is saved until you commit
- Run git status between every step until the model is automatic

GIT

Never commit blind: diff vs diff --staged

```
echo "Day 1: learned the Git model." >> README.md
git diff                # your unstaged change (+/- lines)
git add README.md
git diff                # empty: working dir now matches staging
git diff --staged       # there it is: what you will commit
git commit -m "Add day 1 journal entry"
```

`git diff --staged` is the pre-commit review habit that separates careful engineers from chaotic ones.

GIT

Reading history with git log

```
git log           # full: hash, author, date, message
git log --oneline  # one line per commit: your daily driver
git log -p -1     # full patch of the most recent commit
```

- Every commit has a unique hash (a fingerprint of that snapshot)
- --oneline is what you read day to day
- Press q to leave the pager

Good commits

Do

- Write in the imperative (“Add README setup steps”)
- Keep the first line under ~50 characters
- Small, focused commits (several an hour)
- Say what and why

Avoid

- “stuff”, “changes”, “fixed it”, “final commit”
- Giant commits that mix unrelated work
- Committing without reviewing the diff first

SECURITY

Some files must never be committed

```
# Python
.venv/
__pycache__/

# Secrets: never commit
.env

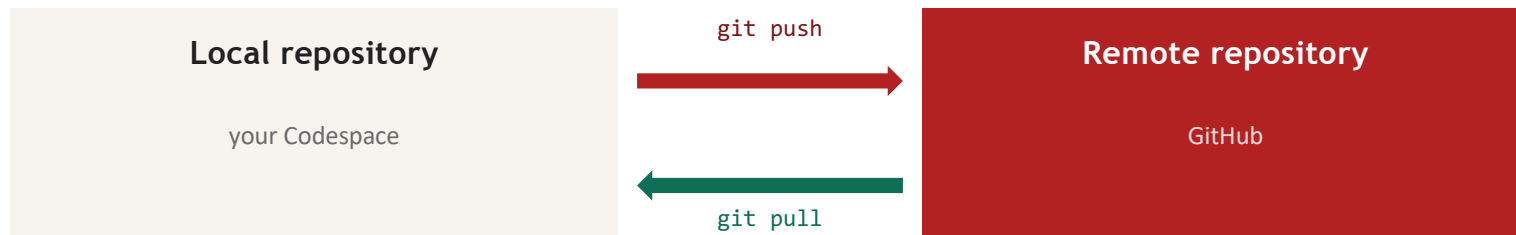
# Logs and build output
*.log
build/
```

The lesson: committing a real API key is a security incident. Bots scan public GitHub for leaked keys within seconds.

The trap: .gitignore only affects untracked files. A secret already committed stays in history; ignoring does not remove it. Rotate the secret.

GIT

Local and remote are two computers



- Your local repo and the GitHub repo are separate copies
- They drift apart until you push or pull; nothing syncs automatically
- Real-world habit: pull before you start working, every time

GIT

Connect and push; the drift demo

```
git remote add origin https://github.com/<you>/claims-journal.git
git remote -v          # origin is a nickname for that URL
git push -u origin main # -u once per branch; then plain push/pull

# someone edits README on github.com, then:
git log --oneline      # your machine has no idea
git pull              # the web commit arrives
git log --oneline      # now it is here
```

-u sets the upstream once; after that, plain `git push` and `git pull`.

GIT

pull, clone, fetch

Command	What it does
git clone <url>	Copy an entire remote repo (history and all) and wire up origin
git pull	Fetch remote commits and merge them into your branch
git fetch	Download remote commits without merging (look before you leap)
git push	Send your local commits to the remote
init + connect when a project starts on your machine; clone when it already exists somewhere. clone is init + remote add + pull, bundled.	

HEADS UP

The 403 on your first push from a blank Codespace

A Codespace created from the blank template has a token scoped to the template, not your new repo, so the first `git push` can fail with “Permission denied” (HTTP 403). Two clean fixes, do not mix them:

Option A (recommended): create the repo on GitHub first, then create the Codespace from that repo (Code, then Codespaces, then Create codespace on main). The token is correct from the start.

Option B: stay in your current Codespace, create a fine-grained Personal Access Token with Contents read and write on the repo, and push once with:

```
git -c credential.helper= push -u origin main
```

GITHUB

GitHub Actions: GitHub does more than version control

- GitHub Actions is GitHub's built-in automation: it runs jobs for you automatically when something happens in your repo — a push, a pull request, or a schedule.
- A workflow is a small YAML file in `.github/workflows/`; its steps run on GitHub's servers, with nothing to install on your own machine.
- For data engineers: run dbt tests and data-quality checks on every pull request, lint SQL/Python, schedule dataset refreshes, and deploy pipelines (CI/CD).

More to come: in Week 4 you'll build a real GitHub Actions CI/CD workflow that runs and tests your dbt models on every pull request — the heart of DevOps & DataOps.

```
# .github/workflows/ci.yml - runs automatically on every pull request
on: pull_request → job: dbt test + data-quality checks → block the merge if red
```

Markdown Mash Time!

Pop Quiz



LAB A

Your repo and your environment

Goal: leave with a working, verified environment you built yourself.

about 30-45 min · pairs

- Create your course repo; launch a Codespace and practice stop and restart (budget)
- Create a venv, install pandas and requests, write requirements.txt
- Add the provided sample claims CSV and write hello_data.py to read it
- Run the edit, stage, commit loop with git status and diffs between steps
- Add a .gitignore; make at least three commits

Deliverable: pushed repo with hello_data.py, the sample CSV, requirements.txt, .gitignore, and the checklist in README.md.

LAB B

Local meets remote

Goal: publish your repo and move work through GitHub.

about 30-45 min · pairs

- Connect your local repo to GitHub and push (handle the 403 with Option A or B)
- Simulate a teammate by editing on github.com, then pull
- Clone a second copy; observe fetch versus pull

Optional stretch (Week 2 on-ramp): branch, open a pull request, review the diff, and merge on your own repo.

Deliverable: published repo on github.com; a clone verified to match; stretch answers noted if attempted.

RECAP

Key takeaways and exit ticket

1

Weeks 1 to 4 are AI-free: fluency and debugging come from writing code by hand

2

Codespaces gives an identical Linux machine: stop it when done, push before you delete

3

One project, one environment, one record: venv + pip baseline; conda and uv do both; uv adds a lock file

4

Git flow: working dir, staging, local, remote. git status is your compass

5

Local and remote are two computers: push sends, pull receives, nothing is automatic

Exit ticket: staging vs committing? · .env still shows after .gitignore, why and fix? · the Codespaces core-hour limit and one habit?